T7. Otras técnicas

Buffer de Profundidad

El **algoritmo del pintor** consiste en dibujar primero los objetos más **lejanos** de la escena según su coordenada z y después **cubrirlos** con los más **cercanos**. Si dos objetos **intersecan**, no sabe cual dibujar primero. Se renderizan todos los puntos de los polígonos, incluso si estos están ocultos en la escena final.

El **Z-Buffer** de profundidad es un buffer que **almacena**, para cada píxel de la pantalla, información por fragmento. Almacena la profundidad (z) del objeto más cercano a la cámara en ese píxel. Tiene el **mismo ancho** y **alto** que el frame-buffer y una profundidad de 16, **24** o 32 bits (a mayor profundidad mayor precisión).

La **prueba de profundidad** consiste en que, cuando se va a **renderizar** un objeto sobre un píxel en el que ya había otro, se **compara** su valor de profundidad (z) con el que ya hay almacenado en el z-buffer. Si el nuevo está **más cerca** de la cámara (menor z), se dibujará y **sustituirá** al antiguo en el z-buffer. Si el nuevo está **más lejos** de la cámara (mayor z) se **descartará**.

En principio, la prueba de profundidad se realiza **después del fragment shader**. Hoy en día la mayoria de GPUs admiten una prueba de profundidad temprana, realizando el test antes del fragment shader para incrementar la performance. Consiste en que, siempre esté claro que un fragmento no será visible (pues está detrás de otros) se descartará prematuramente.

El z-buffer almacena la **profundidad** de un **rango normalizado** entre 0 (más cercano) y 1 (más lejano) para que se compare rápidamente, mientras que en el **view space** la coordenada z está comprendida entre el **near** y el **far** del frustrum.

En realidad, la precisión con la que se almacena la profundidad de los objetos **no es uniforme** en ese rango, los objetos lejanos se comprimen más y los cercanos tienen más detalle, por lo que la relación entre la coordenada z real y el valor almacenado en el buffer (z') sigue una función **no lineal**.

$$z' = \frac{1}{z} \Rightarrow z' = \frac{1/z - 1/near}{1/far - 1/near}$$

$glEnable(GL_DEPTH_TEST)$ activa la prueba de profundidad y $glDisable(GL_DEPTH_TEST)$ desactiva la prueba de profundidad temporalmente, por lo que dibuja según el orden en el que recibe los fragmentos, aunque estén detrás. $glClear(GL_DEPTH_BUFFER_BIT)$ limpia el z-buffer y $glDepthFunc(func)$ modifica los operadores de comparación usados en la prueba de profundidad.

Por defecto, se usa GL_ALWAYS como $func$, que implementa el algoritmo del pintor, pero se pueden usar: GL_NEVER, GL_LESS, GL_EQUAL, GL_LEQUAL, $GL_GREATER$, GL_GEQUAL o $GL_NOTEQUAL$.

- <u>Z-buffer 3.3</u>

En OpenGL 3.3 se puede visualizar el z-buffer accediendo a $gl_FragCoord.z$ en el **fragment shader**, por lo que se puede transformar fácilmente su valor 0-1, en escala de grises.

```
void main() {
FragColor = vec4(vec3(gl_FragCoord.z), 1.0);
 }
```

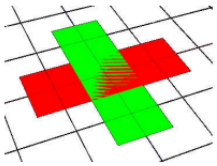
Como es de esperar, los valores obtenidos **no son lineales**, pero se pueden linealizar usando la ecuación antes planteada:

```
#version 330 core
out vec4 FragColor;
float near = 0.1;          //Los podríamos pasar al shader
float far = 10.0;
float no_lineal(float depth) {
 float z = depth * 2.0 - 1.0;
return (2.0 * near * far) / (far + near - z * (far - near)); }
void main() {
float depth = lineal(gl_FragCoord.z) / far;
FragColor = vec4(vec3(depth), 1.0);
}
```

- <u>Z-Fighting</u>

El **z-fighting** es un artefacto visual en el que dos objetos tienen una profundidad tan similar que el z-buffer no tiene **suficiente precisión** para determinar cual está delante, por lo que ambas parecen **cambiar continuamente de orden**, causando efectos extraños. Como la precisión de la profundidad en **zonas cercas** a la cámara es **mayor**, es más común en objetos lejanos. Existen posibles soluciones:

- Aumentar la precisión del z-buffer si es posible.
- Elegir bien el near y far.
- Separar un poco los objetos.



Atenuación

En la **atenuación** la intensidad de iluminación **decrece** a medida que aumenta la distancia a la fuente de luz (salvo para las luces situadas en el infinito).

$$Atenuación = \frac{1}{a + bd + cd^2}$$

donde d es **distancia** entre la **fuente** y el punto que se está **iluminando**. OpenGL establece por defecto $a = 1$, $b = 0$ y $c = 0$, pero estos son valores que se pueden modificar con:

$void\ glLightf(GL_LIGHTX,\ GL_CONSTANT/LINEAR/CUADRATIC_ATTENUATION,\ valor)$.

La atenuación solo influye sobre las componentes de **ambiente** y **difusa** de las luces. Además, puede afectar negativamente al rendimiento.

Niebla

La niebla es un efecto especial utilizado para **difuminar** los objetos según su distancia, haciendo que los más lejanos se mezclen con otro color y parezcan desvanecerse. El efecto niebla se puede aplicar de dos formas:

- Basado en **tablas** (cálculo por píxel), más preciso.
- Basado en **vértices**, más eficiente pero menos preciso.

$glEnable(GL_FOG)$, **activa** la niebla. Para especificar la **densidad**, **color** y **tipo** de niebla se usa $void\ glFogfv(parametro,\ valor)$ donde $parametro$ puede ser: $GL_FOG_DENSITY$, GL_FOG_START, GL_FOG_END, GL_FOG_INDEX o GL_FOG_COLOR.

$glFogi(GL_FOG_MODE,\ GL_LINEAR/EXP/EXP2)$.

`GL_LINEAR` -> $f = \frac{end-z}{end-start}$

`GL_EXP` -> $f = e^{-density*z}$

`GL_EXP2` -> $f = e^{-(density*z)^2}$

MipMapping

El mipmapping consiste en asociar **varias texturas** de **distintas calidades** a la misma superficie para mostrar una de ellas en función de su profundidad. Para usarlo, se debe modificar la función de lectura de la textura.

Se añade una función para cargar una imagen como textura ($GL_TEXTURE_2D$ se especifica que es 2D, 0 es el nivel de detalle, GL_RGB especifica el formato de los colores en formato RGB sin alfa, $width$ y $height$ es el tamaño de la imagen, 0 es el borde de la textura, GL_RGB es el formato de los pixeles, $GL_UNSIGNED_BYTE$ es del tipo de dato de los pixeles, $data$ es un puntero a los datos de la imagen).

$glTexImage2D(GL_TEXTURE_2D,\ 0,\ GL_RGB,\ width,\ height,\ 0,\ GL_RGB,\ GL_UNSIGNED_BYTE,\ data)$;

Una función es una versión más sencilla para generar mipmaps para una textura 2D (GL_RGBA es el formato de los datos que incluye componente de transparencia alfa, ptr es el puntero a los datos de la textura).

$gluBuild2DMipmaps(GL_TEXTURE_2D,\ GL_RGB,\ ancho,\ alto,\ GL_RGBA,\ GL_UNSIGNED_BYTE,\ ptr)$;

Una función establece parámetros de configuración para minimizar una textura ($GL_TEXTURE_MIN_FILTER$ especifica el filtro que se aplicará cuando se reduzca la textura, $GL_LINEAR_MIPMAP_LINEAR$ es el filtro de reducción usando mipmap).

$glTexParameteri(GL_TEXTURE_2D,\ GL_TEXTURE_MIN_FILTER,\ GL_LINEAR_MIPMAP_LINEAR);$

Una función que establece el parámetro de configuración para agrandar una textura ($GL_TEXTURE_MAG_FILTER$ especifica el filtro a aplicar cuando se agrande, GL_LINEAR es el filtro de magnificación con interpolación lineal para que no se pixele).

$glTexParameteri(GL_TEXTURE_2D,\ GL_TEXTURE_MAG_FILTER,\ GL_LINEAR);$

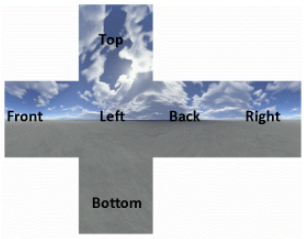
Deberemos modificar la función de lectura de la textura para que se le haga.

```
unsigned char *data = stbi_load(path, &width, &height, &nrComponents, 0);
    if (data)
    {


 gluBuild2DMipmaps(GL_TEXTURE_2D, GL_RGB, width, height, GL_RGB, GL_UNSIGNED_BYTE, data); //con
mimap
        glGenerateMipmap(GL_TEXTURE_2D);
    }
```

SkyBox

La **skybox** es una **caja** que contiene el **escenario** en su interior sobre el que se aplica una textura de cielo. También se puede usar una semicircunferencia o Dome.

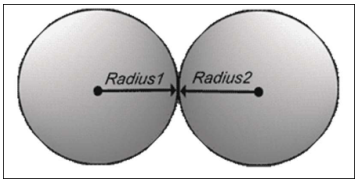


Colisiones

Teóricamente, para saber si dos objetos están colisionando se debe comprobar para cada una de las caras de ambos **si se interseca** con alguna de las del otro. Como esto es muy costoso, se suelen envolver los objetos en una forma geométrica más simple, como una **caja**, una **esfera**, un cilindro o un elipsoide (bounding-box).

- <u>Bounding Spheres</u>

Las boundings spheres (bounding-esferas) es una esfera que engloba el objeto. Se caracteriza por un centro (punto, float $x,\ y,\ z$) y un radio(float).



Dos esferas colisionan si la **distancia** entre sus **centros** es **menor** que la **suma** de sus **radios**.

$$dist = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2 + (z_1 - z_2)^2}$$

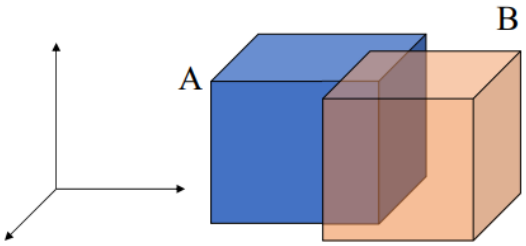
$$if\ (dist\ < (r_1 + r_2)\ \rightarrow colisión)$$

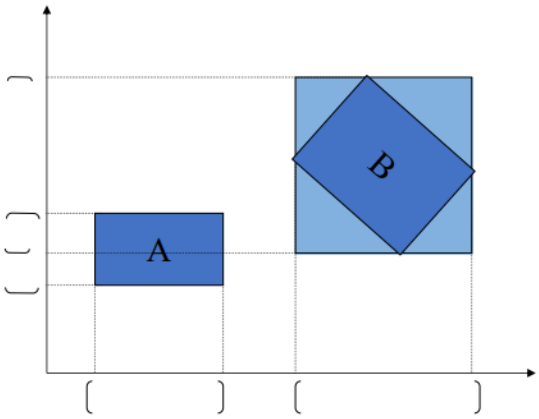
- <u>Bounding Boxes</u>

La **bounding box** es una **caja** que engloba el objeto, se caracteriza por un **punto mínimo minBounds** (punto, float x, y, z) y un **punto máximo maxBounds** (punto, float x, y, z). Hay dos manera de comprobar si las dos cajas colisionan:

- **Axis Aligned Bounding Boxes (AABB)**: se usan **cajas alineadas** con los ejes es el procedimiento más utilizado. Se basa en comprobar si alguna de las **8 esquinas** de la caja está dentro de la caja de otro:

  - AxMin < BxMax and AxMax > BxMin
    - AND
  - AyMin < ByMax and AyMax > ByMin
    - AND
  - AzMin < BzMax and AzMax > BzMin



  El método es más fácil, rápido y barato de calcular.

- **Separated Axis Test (SAT)**: se usan segmentos, triángulos y **cajas no alineadas** con los ejes. Dos objetos no colisionan si existe al menos **un eje de proyección** en el cual al proyectarlo **no se superponen**, objetos disjuntos.